

JavaScript

1. JavaScript Introduction

❖ **Question 1: What is JavaScript? Explain the role of JavaScript in web development.**

ANSWER:-

JavaScript is a high-level, lightweight, and interpreted programming language used to make web pages interactive.

It runs directly inside the browser and allows you to add dynamic behavior to websites.

Role of JavaScript in Web Development

JavaScript is one of the core technologies of the web, along with HTML and CSS:

- **HTML** → Structure (content of the webpage)
- **CSS** → Styling (design, colors, layout)
- **JavaScript** → Behavior (interactivity and dynamic actions)

What JavaScript enables

- Form validation
- Animations and effects
- Dynamic content updates without reloading the page
- Interactive elements (sliders, menus, popups)
- Communication with servers using APIs (AJAX, Fetch)
- Building full web apps (React, Angular, Vue)

❖ **Question 2: How is JavaScript different from other programming languages like Python or Java?**

ANSWER:-

Feature	JavaScript	Python	Java
Type	Scripting language	General-purpose	Object-oriented
Execution	Runs in browser + Node.js	Runs in interpreter	Compiled to bytecode
Typing	Dynamic typing	Dynamic typing	Static typing
Use Case	Web apps, frontend, backend	AI, ML, scripting, automation	Enterprise apps, Android
Syntax	C-style but flexible	Very clean, readable	Strict, verbose
Speed	Fast in browsers	Slower than JS	Fast, high performance

- **JavaScript is primarily for web development**, while Python and Java are used for many different domains.

- Java requires compiling, while JS and Python are interpreted.
- JavaScript runs directly in browsers; Python/Java do not.

❖ **Question 3: Discuss the use of `<script>` tag in HTML. How can you link an external JavaScript file to an HTML document?**

ANSWER:-

Use of `<script>` Tag

The `<script>` tag is used in HTML to:

- Write JavaScript code directly inside HTML
- Link external JavaScript files
- Load JS files at specific positions (head/body)

Inline JavaScript Example

```
<script>  
    console.log("Hello World!");  
</script>
```

External JavaScript File Linking

```
<script src="app.js"></script>
```

Just replace `app.js` with your file name.

Where to place `<script>` tag?

- **At the end of <body>** → Best practice (page loads faster)
- **Inside <head> with defer**

```
<script src="app.js" defer></script>
```

2. Variables and Data Types

❖ Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

ANSWER:-

What are variables?

Variables are containers used to store data values in JavaScript. Example: storing a name, number, or result.

Declaring Variables

JavaScript provides **var**, **let**, and **const**:

1) var

- Old way of declaring variables
- Function-scoped
- Can be re-declared and updated

```
var x = 10;  
x = 20; // allowed  
var x = 30; // allowed
```

2) let

- Block-scoped (modern and preferred)
- Can be updated but **not re-declared in the same block**

```
let a = 5;  
a = 15; // allowed  
// let a = 25; ✗ not allowed
```

3) const

- Block-scoped
- **Cannot be updated or re-declared**
- Must be assigned a value at declaration

```
const pi = 3.14;  
// pi = 3.15; ✗ error
```

❖ Question 2: Explain the different data types in JavaScript with examples.

JavaScript has **8 data types**:

1)String

Represents text.

```
let name = "John";
```

2)Number

Stores integers and decimals.

```
let age = 25;  
let price = 99.99;
```

3)Boolean

Stores **true** or **false**.

```
let isLoggedIn = true;
```

4) Undefined

Variable declared but no value assigned.

```
let x;
```

5) Null

Represents an empty or non-existent value.

```
let data = null;
```

6) BigInt

Used for very large numbers.

```
let bigNum = 12345678901234567890n;
```

7) Symbol

Unique value used mainly in objects.

```
let id = Symbol("userID");
```

8) Object

Collection of key-value pairs.

```
let person = {  
  name: "Alex",  
  age: 30  
};
```

❖ Question 3: What is the difference between `undefined` and `null` in JavaScript?

Feature	<code>undefined</code>	<code>null</code>
Meaning	Variable is declared but not assigned a value	Intentional empty value
Type	<code>undefined</code>	<code>object</code> (due to historical bug)
Set by	JavaScript	Developer
Example	<code>let x;</code>	<code>let y = null;</code>

3.JavaScript Operators

Question 1: What are the different types of operators in JavaScript? Explain with examples.

JavaScript provides many operators, but the four main types are:

1. Arithmetic Operators

Used to perform mathematical operations.

Operator	Meaning	Example
+	Addition	$5 + 3 = 8$
-	Subtraction	$10 - 4 = 6$
*	Multiplication	$6 * 2 = 12$
/	Division	$8 / 2 = 4$
%	Modulus (Remainder)	$10 \% 3 = 1$
**	Exponent	$2 ** 3 = 8$

Example:

```
let a = 10;  
let b = 4;  
console.log(a + b); // 14
```

2. Assignment Operators

Used to assign values to variables.

Operator	Meaning	Example
=	Assign	<code>x = 5</code>
+=	Add and assign	<code>x += 3 → x = x + 3</code>
-=	Subtract and assign	<code>x -= 2</code>
*=	Multiply and assign	<code>x *= 4</code>
/=	Divide and assign	<code>x /= 2</code>

Example:

```
let x = 10;  
x += 5; // x = 15
```

3. Comparison Operators

Used to compare two values; return true or false.

Operator	Meaning	Example
==	Equal to	<code>5 == "5" → true</code>
===	Strict equal	<code>5 === "5" → false</code>

<code>!=</code>	Not equal	<code>10 != 5 → true</code>
<code>></code>	Greater than	<code>7 > 4 → true</code>
<code><</code>	Less than	<code>3 < 7 → true</code>
<code>>=</code>	Greater or equal	<code>5 >= 5 → true</code>
<code><=</code>	Less or equal	<code>4 <= 6 → true</code>

4. Logical Operators

Used to combine multiple conditions.

Operator	Meaning	Example
-----	-----	
<code>&&</code>	Logical AND	<code>(5 > 3 && 10 > 7) → true</code>
<code> </code>	Logical OR	<code>(5 > 3 10 < 7) → true</code>
<code>!</code>	Logical NOT	<code>!(5 > 3) → false</code>

Question 2: What is the difference between `==` and `===` in JavaScript?

1. `==` (Loose Equality)

- Compares values only
- Performs type conversion

Example:

```
5 == "5"; // true
```

JavaScript converts "5" to number 5.

2. === (Strict Equality)

- Compares value AND data type
- No type conversion

Example:

```
5 === "5"; // false
```

Simple Explanation

Operator	Checks	Example
==	Value only	"5" == 5 → true
===	Value + Type	"5" === 5 → false

4. Control Flow (If-Else, Switch)

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

What is Control Flow?

Control flow in JavaScript refers to the order in which statements are executed in a program.

Normally, code runs top to bottom, but with control flow statements (if-else, loops, switch), you can change the order based on conditions.

How if-else works

`if-else` checks conditions and runs different blocks of code based on whether the condition is true or false.

Syntax

```
if (condition) {  
    // runs if condition is true  
} else {  
    // runs if condition is false  
}
```

Example

```
let age = 18;  
  
if (age >= 18) {  
    console.log("You are eligible to  
vote");  
} else {
```

```
    console.log("You are not eligible  
to vote");  
}
```

Explanation

- If the condition `age >= 18` is true, the first block runs.
- If the condition is false, the `else` block runs.

Question 2: Describe how switch statements work in JavaScript. When should you use a switch instead of if-else?

How a switch statement works

A switch checks the value of an expression and matches it with different cases.

If a match is found, that block of code runs.

Syntax

```
switch(expression) {  
    case value1:  
        // code block  
        break;  
  
    case value2:
```

```
        // code block
        break;

    default:
        // code if no case matches
}
```

Example

```
let day = 3;

switch(day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid day");
}
```

Explanation

- If day equals 3, it matches case 3, so "Wednesday" is printed.
- break stops the switch from checking further cases.

When to use switch instead of if-else?

Use switch when:

- You have multiple conditions based on the same variable
- You are checking many fixed values like:
 - days
 - months
 - menu options
 - grades
- The code becomes cleaner and more readable than long if-else chains

Example of bad if-else chain

```
if (color === "red") { }  
else if (color === "blue") { }  
else if (color === "green") { }
```

Switch is cleaner

```
switch(color) {  
  case "red":  
  case "blue":  
  case "green":
```



```
}
```

5.Loops (For, While, Do-While)

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Loops are used to repeat a block of code multiple times.

1. for Loop

Used when you know exactly how many times you want to run the loop.

Syntax

```
for (initialization; condition;
    increment) {
    // code
}
```

Example

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

Output: 1 2 3 4 5

2. while Loop

Used when the number of repetitions is unknown and depends on a condition.

Syntax

```
while (condition) {  
    // code  
}
```

Example

```
let i = 1;  
  
while (i <= 5) {  
    console.log(i);  
    i++;  
}
```

3. do-while Loop

This loop runs at least once, even if the condition is false.

Syntax

```
do {  
    // code
```

```
} while (condition);
```

Example

```
let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

Question 2: What is the difference between a while loop and a do-while loop?

Feature	while Loop	do-while Loop
Condition check	Checked before loop runs	Checked after loop runs
Minimum execution	May run 0 times	Runs at least 1 time
Syntax order	condition → body	body → condition

Simple Example Difference

while

```
let x = 5;

while (x < 5) {
```

```
    console.log("Hello");  
}
```

Condition false → loop never runs

do-while

```
let x = 5;
```

```
do {  
    console.log("Hello");  
} while (x < 5);
```

Output: "Hello" (runs once even though condition is false)

6.Functions

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

What are functions?

A function in JavaScript is a block of reusable code designed to perform a specific task.

Instead of repeating code, we write a function and call it whenever needed.

Function Declaration Syntax

```
function functionName() {  
    // code to execute  
}
```

Calling (executing) a Function

```
functionName();
```

Example

```
function greet() {  
    console.log("Hello!");  
}
```

```
greet();    // calling the function
```

Question 2: What is the difference between a function declaration and a function expression?

1. Function Declaration

- Defined using the `function` keyword with a name
- Hoisted → can be called *before* it is defined

Example:

```
function add(a, b) {  
    return a + b;  
}
```

2. Function Expression

- Function is stored in a variable
- Can be anonymous (no name)
- Not hoisted → cannot be used before definition

Example:

```
const add = function(a, b) {  
    return a + b;  
};
```

Key Differences

Feature	Function Declaration	Function Expression
Name	Always named	Can be anonymous
Hoisting	Hoisted	Not hoisted
When defined	At compile time	When the code runs
Usage	Can call before definition	Must call after definition

Question 3: Discuss the concept of parameters and return values in functions.

Parameters

- Variables listed inside the function definition
- They act like placeholders for input values

Example:

```
function add(a, b) { // a and b are
  parameters
    return a + b;
}
```

Arguments

- Actual values passed to the function when calling it

Example:

```
add(5, 10); // 5 and 10 are arguments
```

Return Values

A function can return a value using the return keyword.

Example:

```
function multiply(x, y) {  
    return x * y;  
}
```

```
let result = multiply(4, 5);  
console.log(result); // 20
```

Why return is used?

- Gives a result back to the code
- Stops function execution
- Allows storing calculation results

7.Arrays

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

What is an Array?

An array in JavaScript is a special type of object used to store multiple values in a single variable.

These values can be of different data types (number, string, boolean, objects, etc.).

Why arrays?

- Store a list of items
- Access elements using indexes (starts from 0)

How to Declare and Initialize an Array

1. Using Square Brackets (most common)

```
let fruits = ["apple", "banana",  
"orange"];
```

2. Using the Array Constructor

```
let numbers = new Array(10, 20, 30);
```

3. Empty Array Initialization

```
let emptyArray = [];
```

Accessing Array Elements

```
console.log(fruits[0]); // "apple"
```

Question 2: Explain push(), pop(), shift(), and unshift() methods used in arrays.

These four methods help add or remove elements from arrays.

1. push() – Add element at the end

Definition:

Adds one or more items to the end of the array.

Example:

```
let fruits = ["apple", "banana"];  
fruits.push("orange");
```

```
console.log(fruits); // ["apple",  
"banana", "orange"]
```

2. pop() – Remove element from the end

Definition:

Removes the last element from the array.

Example:

```
let fruits = ["apple", "banana",  
"orange"];  
fruits.pop();  
  
console.log(fruits); // ["apple",  
"banana"]
```

3. shift() – Remove element from the beginning

Definition:

Removes the first element of the array.

Example:

```
let fruits = ["apple", "banana",  
"orange"];  
fruits.shift();  
  
console.log(fruits); // ["banana",  
"orange"]
```

4. unshift() – Add element at the beginning

Definition:

Adds one or more items to the start of the array.

Example:

```
let fruits = ["banana", "orange"];  
fruits.unshift("apple");
```

```
console.log(fruits); // ["apple",  
"banana", "orange"]
```

Summary Table

Method	Action	Position
push()	Add	End
pop()	Remove	End
shift()	Remove	Beginning
unshift()	Add	Beginning

8.Objects

Question 1: What is an object in JavaScript? How are objects different from arrays?

What is an Object?

An object in JavaScript is a collection of key–value pairs. Each key is called a property, and it stores a value (number, string, array, function, etc.).

Example of an object

```
let person = {  
  name: "Rahul",  
  age: 22,  
  city: "Ahmedabad"  
};
```

How Are Objects Different from Arrays?

Feature	Object	Array
Structure	Key–value pairs	Ordered list of values
Access	Using keys (name, age)	Using indexes (0, 1, 2)
Order	Not necessarily ordered	Always ordered
Usage	Real-world entities (person, car)	Lists (fruits, numbers)
Keys	Custom names	Only numeric indexes

Simple difference

- Objects → store information with names
Example: `person.name`

- Arrays → store lists using index numbers
Example: `fruits[0]`

Question 2: Explain how to access and update object properties using dot notation and bracket notation.

Objects have properties, and there are two ways to access or update them:

1. Dot Notation

Simplest and most common way.

Access

```
console.log(person.name) ;
```

Update

```
person.age = 23;
```

2. Bracket Notation

Used when:

- Property name has spaces
- Property name starts with a number
- Property name is in a variable

Access

```
console.log(person["city"]);
```

Update

```
person["city"] = "Mumbai";
```

✓ Example with Both Notations

```
let person = {  
  name: "Rahul",  
  age: 22,  
  city: "Ahmedabad"  
};
```

```
// Dot notation
```

```
console.log(person.name);    // Rahul  
person.age = 23;
```

```
// Bracket notation
```

```
console.log(person["city"]); // Ahmedabad  
person["city"] = "Mumbai";
```

9.JavaScript Events

Question 1: What are JavaScript events? Explain the role of event listeners.

What are JavaScript Events?

Events are actions or occurrences that happen in the browser and can trigger JavaScript code.

Examples of events:

- Clicking a button → `click` event
- Pressing a key → `keydown` event
- Moving the mouse → `mousemove` event
- Loading a webpage → `load` event
- Typing in an input field → `input` event

Role of Event Listeners

Event listeners are used to detect and respond to these events.

They allow JavaScript to:

- Wait for a user action
- Run a function when the event happens
- Make web pages interactive

For example:

- Show a popup when a button is clicked
- Validate a form when a user types
- Change colors when user hovers over something

Event listeners connect user actions with JavaScript functions.

Question 2: How does the `addEventListener()` method work in JavaScript? Provide an example.

What is `addEventListener()`?

`addEventListener()` is a method used to attach an event to an HTML element.

Syntax

```
element.addEventListener(event,  
function);
```

Meaning

- event → the event type (e.g., "click", "mouseover", "keydown")

- function → what should happen when the event occurs

Example

HTML

```
<button id="btn">Click Me</button>
```

JavaScript

```
let button =  
document.getElementById("btn");
```

```
button.addEventListener("click",  
function() {  
    console.log("Button was  
clicked!");  
});
```

Explanation

- It finds the button using `getElementById`
- `addEventListener()` listens for a "click" event
- When clicked → it runs the function and prints the message

10. DOM Manipulation

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

What is the DOM?

The DOM (Document Object Model) is a tree-like structure created by the browser to represent an HTML document.

- Every HTML element becomes a node (object)
- JavaScript can access and modify these nodes
- DOM allows JavaScript to interact with the webpage dynamically

Think of the DOM as a bridge between JavaScript and HTML.

How JavaScript Interacts with the DOM

JavaScript can:

- Select HTML elements
- Change content, styles, attributes
- Add or remove elements
- Handle events (clicks, typing, form submit)

Example

```
document.getElementById("title").innerHTML = "Hello JavaScript!";
```

This finds the element with id "title" and changes its text.

Question 2: Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.

JavaScript provides multiple methods to select HTML elements.

1. `getElementById()`

- Selects a single element by its id
- Returns one element only
- Fastest method

Example

```
let title =  
document.getElementById("mainTitle");
```

2. `getElementsByClassName()`

- Selects all elements with a given class
- Returns an `HTMLCollection` (array-like)
- You need to use index to access items

Example

```
let items =  
document.getElementsByClassName("box")  
;  
console.log(items[0]);    // first  
element with class "box"
```

3. `querySelector()`

- Selects the first matching element using CSS selectors
- Very flexible

You can select by:

- ID → `#id`
- Class → `.class`
- Tag → `tagname`
- Attributes → `[type="text"]`
- Nested selectors

Example

```
let element =  
document.querySelector(".container");
```

Summary Table

Method	Returns	Selector Type	Example
<code>getElementById()</code>	Single element	ID only	<code>document.getElementById("name")</code>
<code>getElementsByName()</code>	HTMLCollection	Class only	<code>document.getElementsByName("item")</code>
<code>querySelector()</code>	First matching element	CSS selector	<code>document.querySelector(".item")</code>

11 .JavaScript Timing Events (setTimeout, setInterval)

Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?

JavaScript provides two important timing functions:

1. setTimeout()

What it does:

- Runs a function once
- After a specified delay (in milliseconds)

Use case:

- Show a message after 3 seconds
- Delay animations
- Load content after some time

Syntax:

```
setTimeout(function, delay);
```

Example delay:

- 1000 ms = 1 second
- 2000 ms = 2 seconds

2. setInterval()

What it does:

- Runs a function repeatedly
- At fixed time intervals

Use case:

- A clock that updates every second
- Auto-refresh data
- Repeating animations

Syntax:

```
setInterval(function, interval);
```

✓ Summary

Function	Runs	Timing
setTimeout()	Once	After delay

Function	Runs	Timing
----------	------	--------

<code>setInterval()</code>	Repeatedly	Every interval
----------------------------	------------	----------------

Question 2: Example of using `setTimeout()` to delay an action by 2 seconds

Example Code

```
setTimeout(function() {  
    console.log("This message appears  
after 2 seconds");  
}, 2000);
```

Explanation

- The function inside `setTimeout()` will run after 2000 ms (2 seconds).
- It prints the message after the delay.

11. JavaScript Error Handling

- ❖ Question 1: What is error handling in JavaScript?
Explain the try, catch, and finally blocks with an example.

What is Error Handling?

Error handling in JavaScript is the process of detecting and managing errors in a program so the script doesn't unexpectedly stop.

JavaScript provides try-catch-finally to handle errors safely.

1.try block

- Contains the code that may produce an error
- If an error occurs → control goes to catch

Example:

```
try {  
    // risky code  
    let result = x + 10;    // x is not  
defined → error  
}
```

2. catch block

- Runs only if an error occurs in try block
- Receives the error object

Example:

```
catch(error) {  
    console.log("An error occurred: " + error.message);  
}
```

3. finally block

- Runs always, whether an error occurs or not
- Used for cleanup tasks (closing connections, stopping loaders, etc.)

Example:

```
finally {  
    console.log("Execution complete");  
}
```

➤ Complete Example

```
try {  
    let result = x + 10;    // x is not  
    defined → error
```

```
}  
catch (error) {  
    console.log("Error caught: " +  
error.message);  
}  
finally {  
    console.log("This will run no  
matter what.");  
}
```

Output

```
Error caught: x is not defined  
This will run no matter what.
```

❖ Question 2: Why is error handling important in JavaScript applications?

Error handling is important because it:

1. Prevents the application from crashing

If an error occurs and is not handled, the program may stop working.

2. Improves user experience

Instead of showing a blank page or stopping, the app can show:

- error messages
- fallback content
- retry options

3. Makes debugging easier

Developers can:

- log errors
- identify where the issue happened

4. Helps maintain smooth flow

Even when errors occur, the rest of the application can continue running.

5. Ensures reliability

Proper error handling makes the app:

- stable
- secure
- professional